🖥️ ctOS Dev Portfolio - Technical Specification

The Next-Generation Developer Interface & System Portfolio

An immersive, high-fidelity developer portfolio designed as a terminal emulator with a "cyber-security" aesthetic. Built on the core philosophy of non-linear exploration and system-driven storytelling.

📋 Table of Contents

🎯 Overview

The **ctOS Dev Portfolio** transcends the traditional "static grid" portfolio by transforming the browsing experience into an interactive operating system. The application serves as a centralized hub for all professional projects, experimental builds, and technical documentation, accessible through both visual and command-line interfaces.

Key objectives:

- **Immersive Interaction**: Users explore projects via a virtual terminal and filesystem.
- **Unified Registry**: Centralized project metadata for multi-view rendering (Archive vs. Sandbox).
- **System Realism**: Mimicking terminal behaviors, boot sequences, and glitch effects.

🎨 Aesthetic Philosophy

The design system is heavily influenced by high-tech "Hacker" interfaces (Watch Dogs' ctOS, Cyberpunk 2077) using a **Cyber-Industrial** aesthetic.

- **Glassmorphism**: Transparent, frosted-glass containers with vibrant purple and green accents.
- **Dynamic Glitch Engine**: Custom-built DedSec-inspired glitch effects that can be triggered system-wide via CLI.
- **CRT Post-Processing**: Subtle scanlines, noise, and RGB separation to simulate high-end monitoring hardware.
- **Micro-Animations**: Extensive use of Framer Motion for layout transitions, hover states, and "data-streaming" entrance animations.

✨ Core Features

💻 Terminal Emulator

- **Real-time CLI**: A fully functional command registry supporting standard Unix-like commands.

- **Command History**: Navigate previous inputs using arrow keys.
- **Typed Responses**: Simulated latency in text output for a "log-streaming" feel.

📁 Virtual Filesystem

- **Directory Traversal**: Explore project files through `cd` and `ls`.
- **Data Inspection**: Use `cat` to read detailed project specifications directly in the terminal.
- **Hierarchical Structure**: Deep-nested directories for `archive`, `uplink`, and `root`.

🧪 Innovation Lab (Sandbox)

- **Experimental Workspace**: A dedicated section for "In-Development" or "Draft" projects.
- **IDE Simulator**: A built-in code editor (Monaco-like) and production simulation environment.
- **Deep Scan View**: High-fidelity technical breakdowns for draft projects with private repositories.

📡 Comm Uplink

- **Secure Messaging**: Integrated email drafting and contact gateway.
- **Encrypted Channels**: Visual feedback indicating "secure uplink establishment".

🛠️ Tech Stack

Frontend Core

- **Framework**: Next.js 15+ (App Router & Server Actions)
- **Runtime**: React 19 (Stable)
- **Language**: TypeScript (Strict Mode)
- **Styling**: Tailwind CSS 4.0 (Modern utility-first architecture)

Animation & Motion

- **Framer Motion**: Standardized micro-interactions and layout transitions.
- **GSAP**: High-performance scrolling and complex SVG path animations.
- **CSS-in-JS**: Dynamic glitch keyframes and scanline filters.

Infrastructure & UI

- **Lucide React**: Vector-scaled iconography.
- **System Components**: Custom-built `SystemCard`, `FloatingTerminal`, and `ProjectLinkController`.
- **State Management**: React Context API for terminal session and glitch state orchestration.

📐 Core Architecture

The project follows a modular "System // Module" architecture:

```
frontend/
├── app/                  # Route-based page modules (Dashboard, Archive, Sandbox, Uplink)
├── components/           # Reusable UI primitives
│   ├── archive/          # Archive-specific grid and detail components
│   ├── sandbox/          # IDE simulator and file explorer components
│   └── ui/               # Core brand components (SystemCard, LinkController)
├── lib/                  # System logic and data registries
│   ├── filesystem.ts     # The Virtual Filesystem (VFS) Tree structure
│   ├── projects.ts       # The Centralized Project Metadata Registry
│   ├── commands.ts       # CLI Command Logic and Execution Engine
```

```
│   └── terminal.ts       # Terminal state and history management
└── public/               # Static documentation and system assets
```

🔌 CLI Command Registry

The system supports a growing list of operational commands:

Command	Usage	Description
`neofetch`	`neofetch`	Displays system specs and operator avatar.
`projects`	`projects [--all]`	Lists all high-tier archive projects.
`ls`	`ls [dir]`	Lists contents of the current virtual directory.
`cd`	`cd [dir]`	Navigates the virtual filesystem.
`cat`	`cat [file.txt]`	Reads project files and documentation.
`clear`	`clear`	Flushes the terminal buffer.

📂 Virtual Filesystem (VFS)

The VFS is a JSON-based tree structure defined in `lib/filesystem.ts`. It maps virtual files to a "Document Object" which can be rendered by the `cat` command.

VFS Nodes:

- `/root`: System core files.
- `/archive`: Operational records and project specifications.
- `/uplink`: Contact links and communication protocols.

🧬 Project Management

All projects are managed through a single source of truth: `lib/projects.ts`.

- **Categorization**: Primary Deployments (Major) vs. Minor Projects.
- **Draft Status**: Projects marked as `isDraft: true` are automatically moved to the **Innovation Lab** and hidden from the standard Archive grid.
- **Dynamic Docs**: Projects retrieve their documentation (`.md` or `.pdf`) dynamically based on their `docUrl` property.

🏗️ Installation & Deployment

Prerequisites

- Node.js 20+
- npm (or yarn/pnpm)

Setup

1. **Clone Repo**

```
git clone https://github.com/pd241008/ctOS-Dev-Portfolio
cd ctOS-Dev-Portfolio
```

2. **Install**

```
npm install
```

3. **Boot System**

```
npm run dev
```

Deployment

Optimized for **Vercel** with Edge Runtime support.

```
# Production Build
npm run build
```

🤝 Acknowledgments

- **Watch Dogs (Ubisoft)**: For the "ctOS" aesthetic inspiration.
- **Vercel AI SDK**: For potential future LLM-integrated terminal agents.
- **The Open Source Community**: For the robust toolsets powering this interface.

OPERATOR: prathmesh_desai LAST_SYNC: 2026.03.16.2250 STATUS: SYSTEM_READY